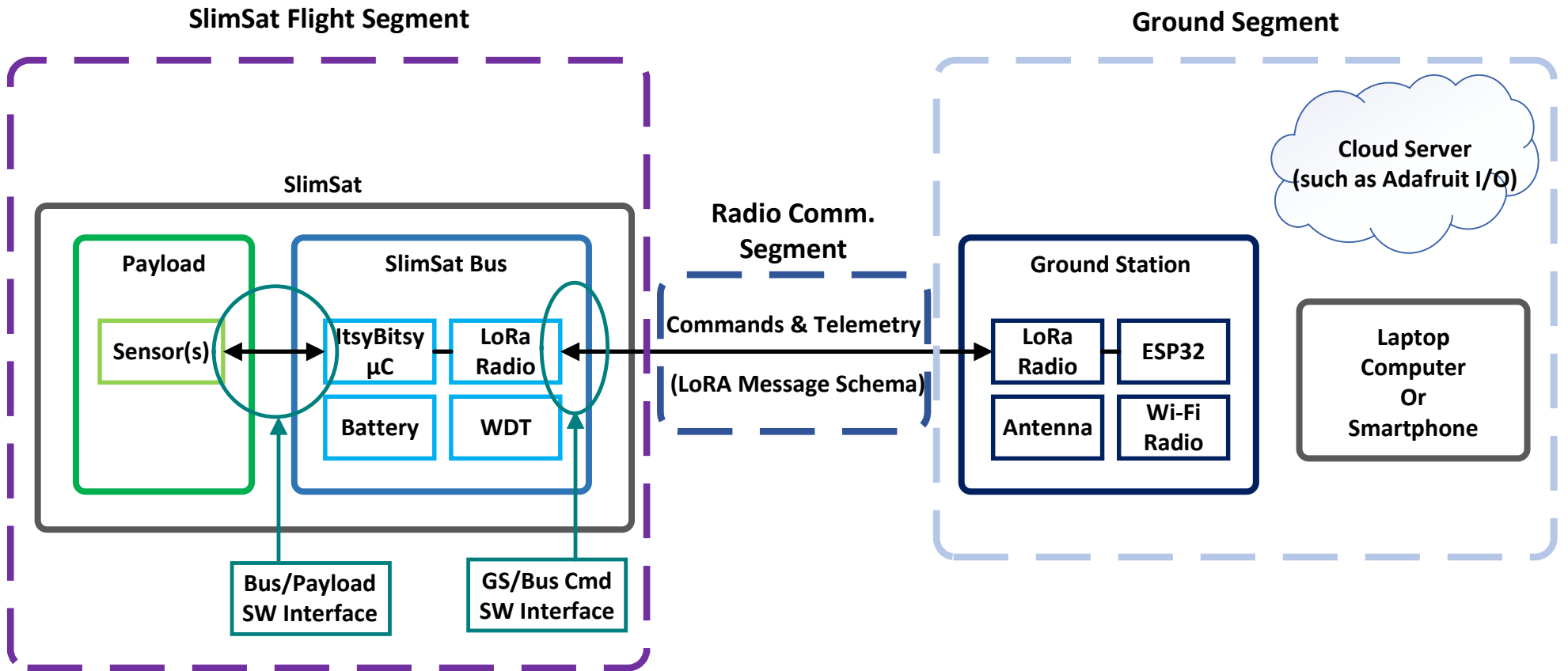


Breakdown of SlimSat System-Level Responsibilities



Payload

HW Bus/PL IF: U of L., Nick
SW: Eric & Team

Sensors

HW: SlimSat Teams
SW: SlimSat Teams

SlimSat Bus

HW: U of L., Nick & Team
SW: Eric & Team

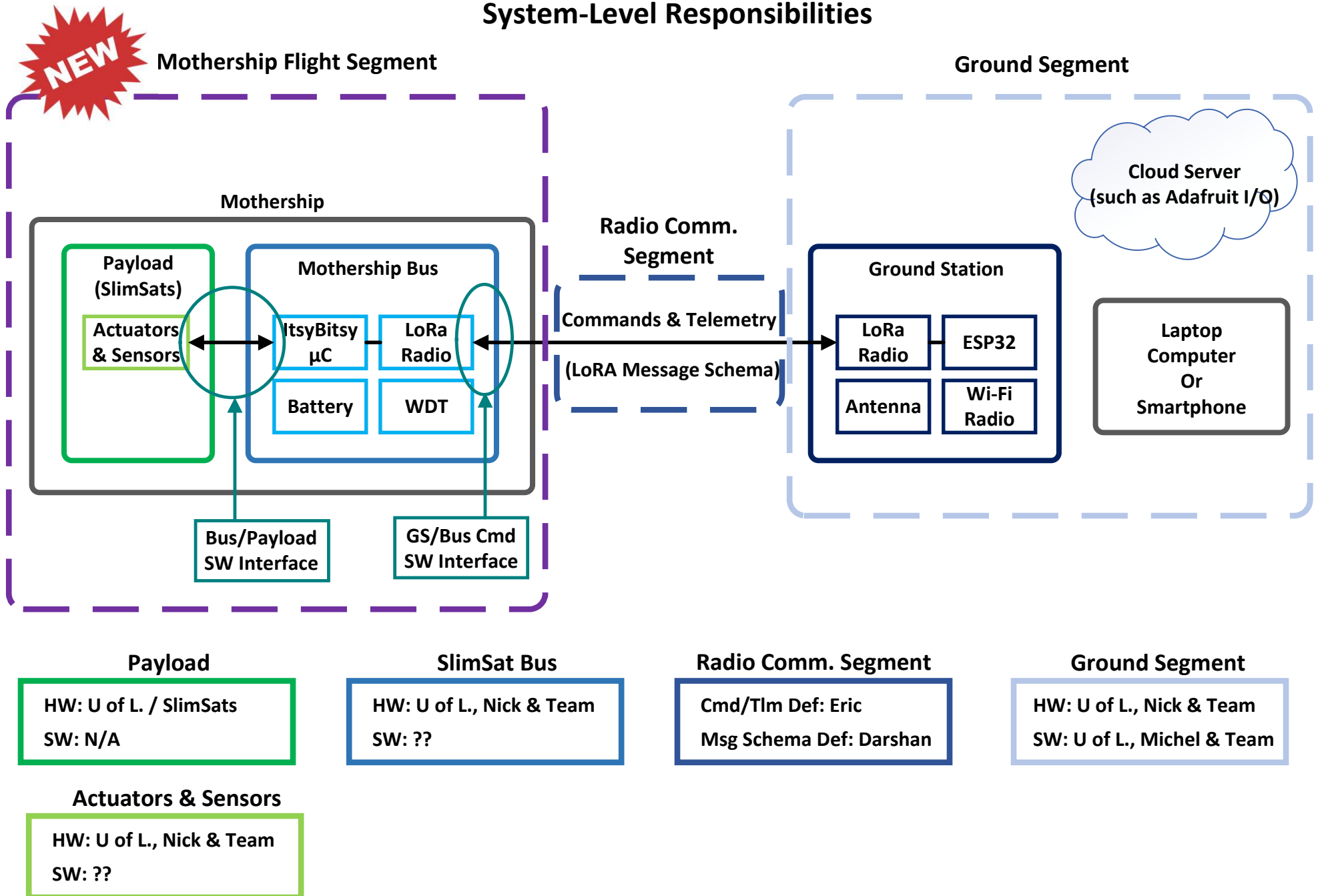
Radio Comm. Segment

Cmd/Tlm Def: Eric
Msg Schema Def: Darshan

Ground Segment

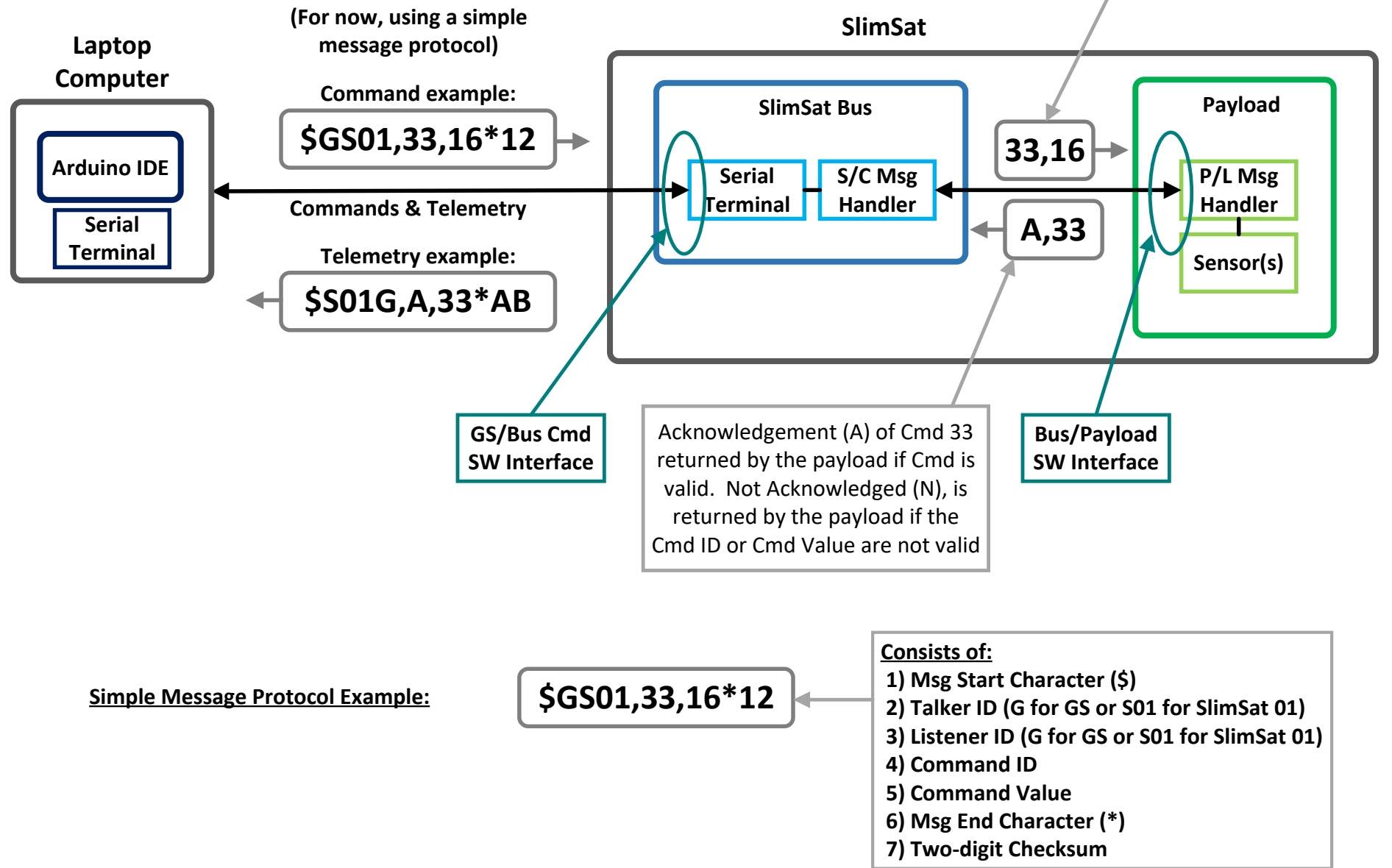
HW: U of L., Nick & Team
SW: U of L., Michel & Team

Breakdown of Mothership System-Level Responsibilities



Context Reminder of the Development SlimSat System and How it operates:

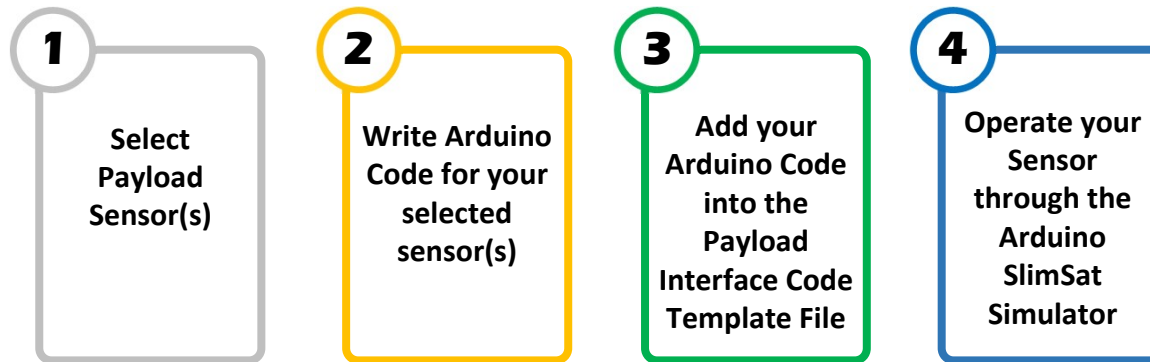
SlimSat System Interfaces



SlimSat Payload Development Process

How to enable student SlimSat Teams to develop functional payloads?

Currently putting together a Payload Developer's Guide, which includes a Ping Sensor Payload example, for Student Teams covering the following 4 Step process:



Preliminary Payload Class (C++ Code) Definition

3

Add your
Arduino Code
into the
Payload
Interface Code
Template File

Sensor code that generally goes
into the Arduino "setup"
function gets put into the
InitializePayload function instead

Sensor code that generally goes
into the Arduino "loop" function
gets put into the
performPayloadOperationIteration
function instead

Preliminary

```
+-----+  
| <<enumeration>> |  
| Payload_State    |  
+-----+  
| STOPPED = 0      |  
| RUNNING = 1      |  
+-----+
```

The Payload Class defines the software
structure and public interface which shall
be conformed to enable one or more
sensors to be integrated into the SlimSat

```
+-----+  
| Payload           |  
+-----+  
| - pl_state: Payload_State  
| - pl_msg_buffer: char[PAYLOAD_MSG_BUFFER_LENGTH]  
+-----+  
| + Payload()  
| + print(): void const  
| + initializePayload(): void  
| + initializePayloadMessageBuffer(): void  
| + handlePayloadCommand(cmd_id: uint8_t, cmd_val: uint8_t): uint8_t  
| + getPayloadState(): uint8_t  
| + getPayloadDataStr(): char*  
| + performPayloadOperationIteration(): uint8_t  
+-----+
```

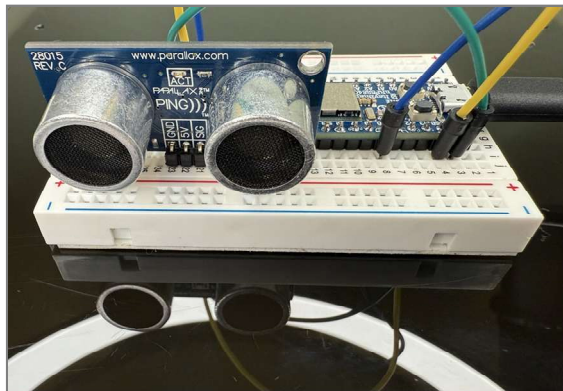
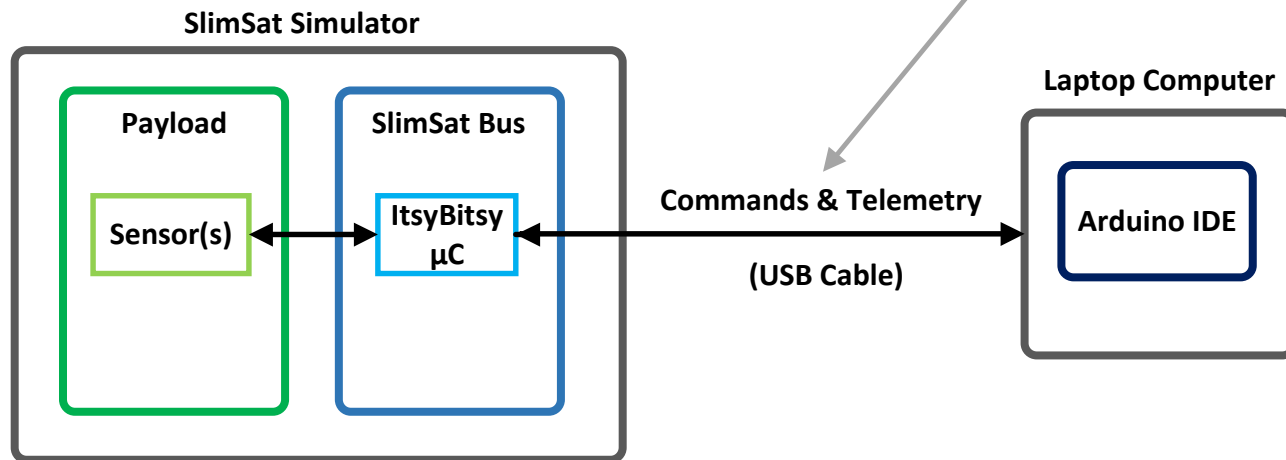
Additionally, the Payload Class
provides flexibility to add any
code required to interface to
SlimSat team's payload sensor(s)

Note: From experience with my own team, I
anticipate a fair amount of training and
guidance will be required to help students
make this jump to work with Object
Oriented Programming (OOP), such as the
Payload Interface Code Template file.
(It's still C++ code, just organized differently)

SlimSat Simulator Setup

Handles Payload commands and
almost all S/C Bus commands
(though no SlimSat HW to affect)

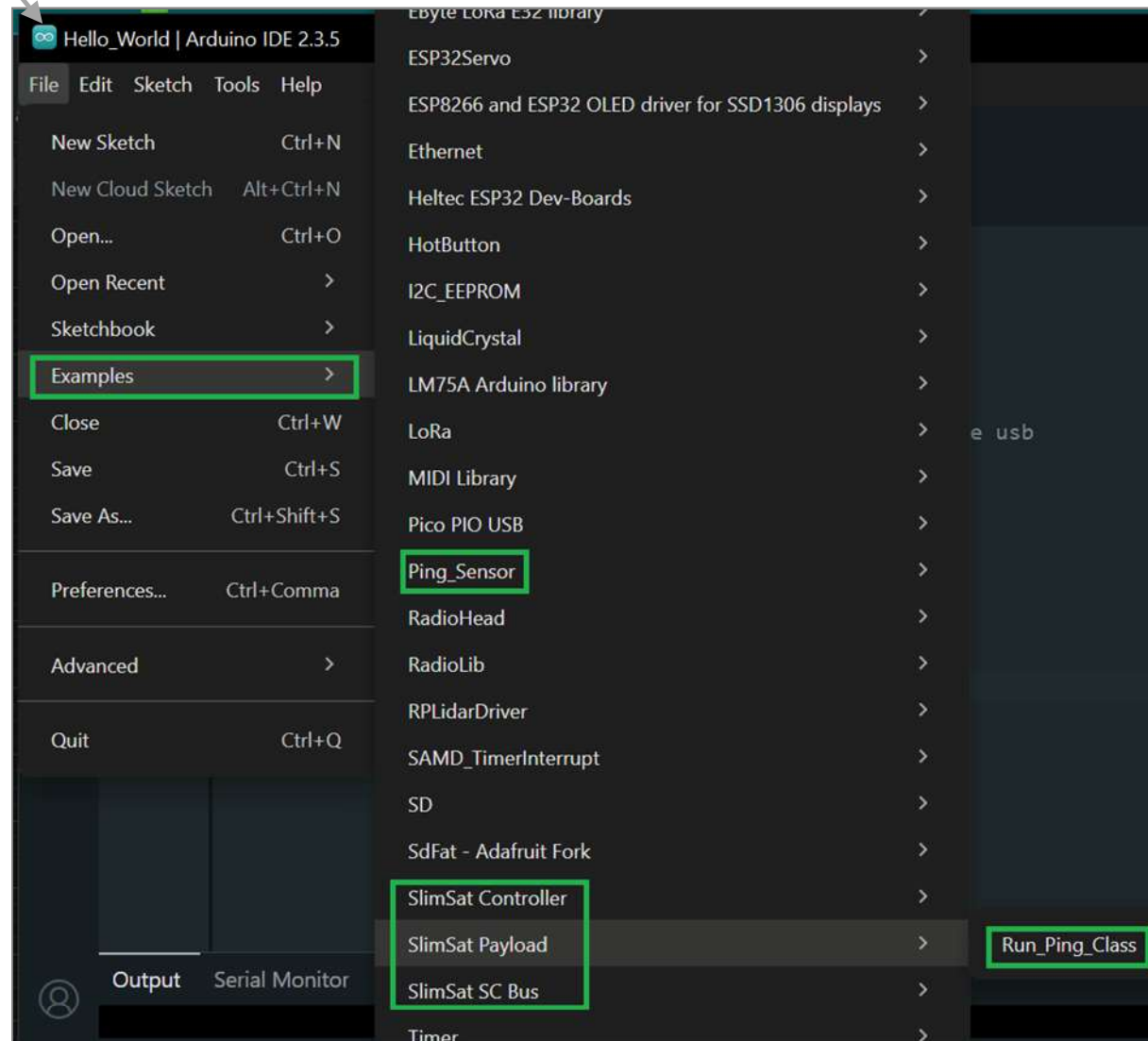
Uses simple command ID and command value
input pairs, such as:
"18,2000"
(Cmd S/C Bus to call the Payload every 2 seconds)



Ping Payload Example Commands	Command ID
Ping Payload	30
Get Payload State	31
Get Number Measurements to Take	32
Set Number Measurements to Take	33
Take Round of Measurements	34
Get Raw Measurements	35
Get Processed Measurement Data	36
Print Measurements	37
--	38
--	39

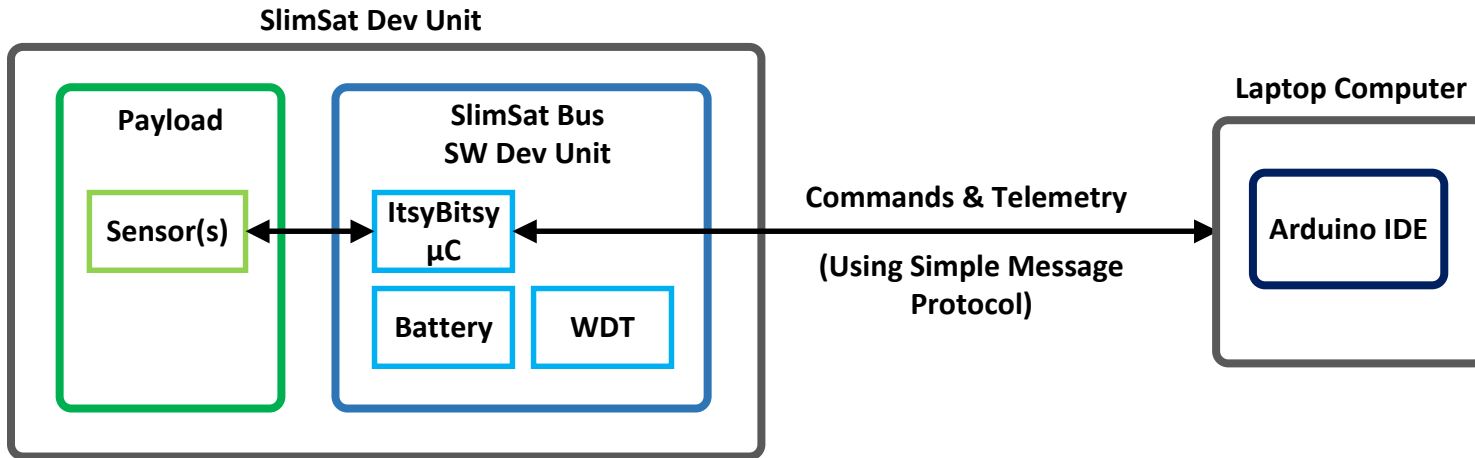
SlimSat examples made easy for students, can be found in the Arduino IDE Examples after code downloaded and installed

SlimSat Examples Accessible from Arduino IDE Examples



SlimSat SW Development & Testing – Nex Steps

Step 1: Use a simple message protocol using readable ASCII text (largely borrowed from NMEA 0183)



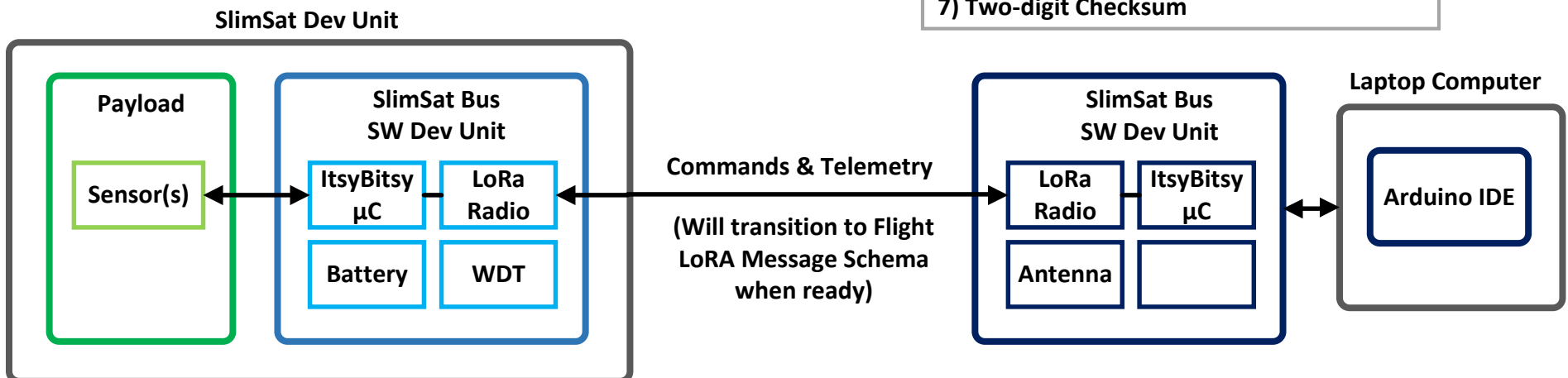
Simple Message Protocol Example:

\$GS01,18,2000*AB

Consists of:

- 1) Msg Start Character (\$)
- 2) Talker ID (G for GS or S01 for SlimSat 01)
- 3) Listener ID (G for GS or S01 for SlimSat 01)
- 4) Command ID
- 5) Command Value
- 6) Msg End Character (*)
- 7) Two-digit Checksum

Step 2:

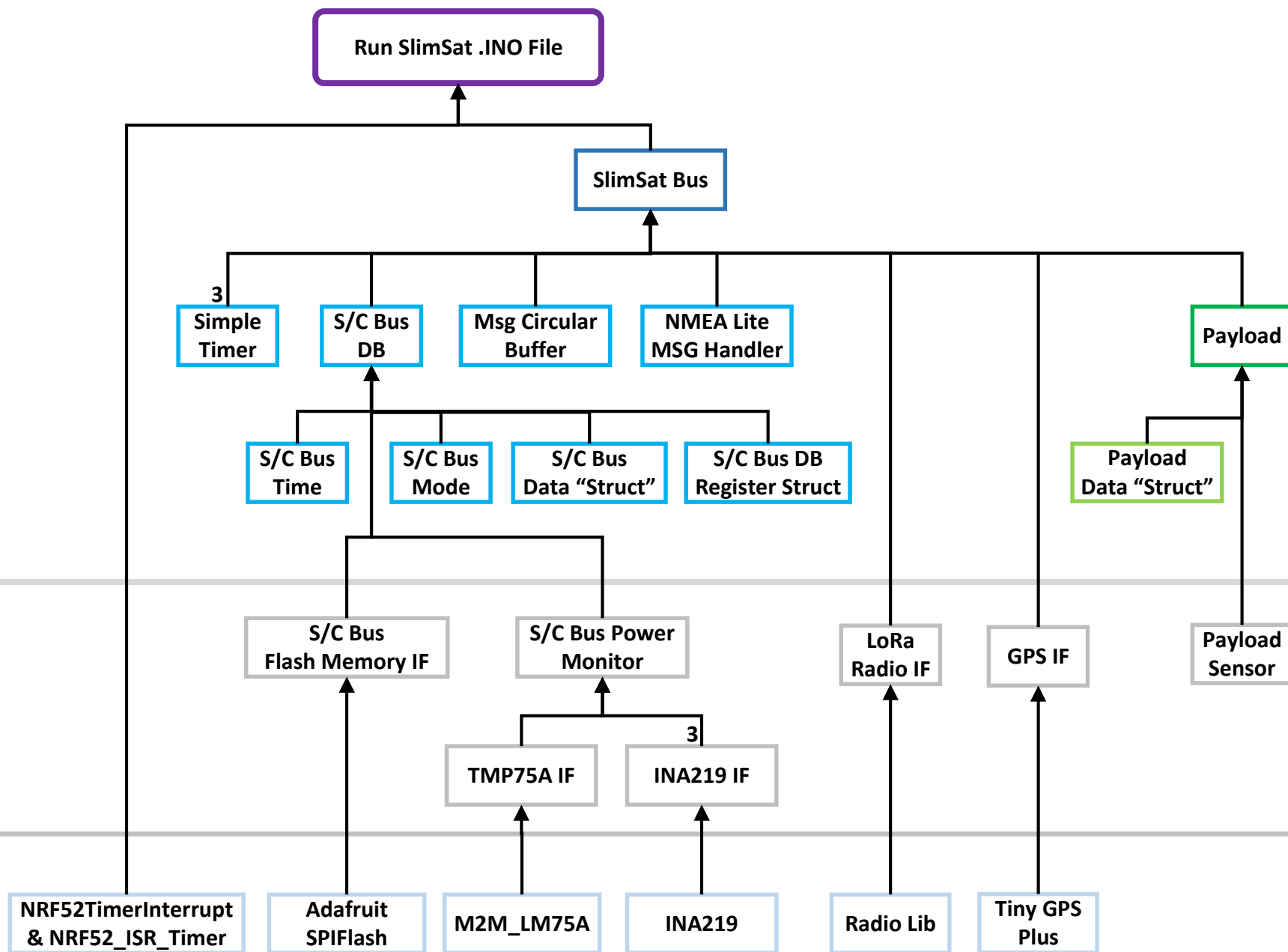


SlimSat Software Class Architecture

SlimSat
Software
Libraries
(No HW
Required)

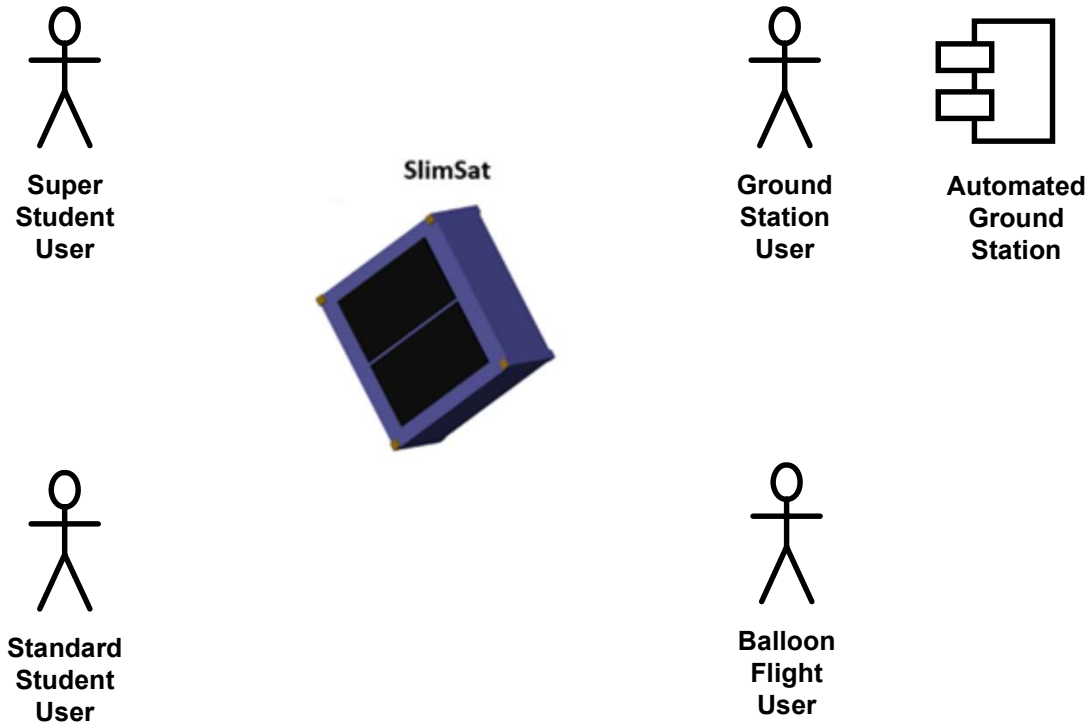
SlimSat
Software
Libraries
(Wrapper
Classes
Requiring
HW)

Arduino
Software
Libraries
(Requiring
HW)



CAPE-Twiggs HSSSI-26 Project

SlimSat System User Analysis

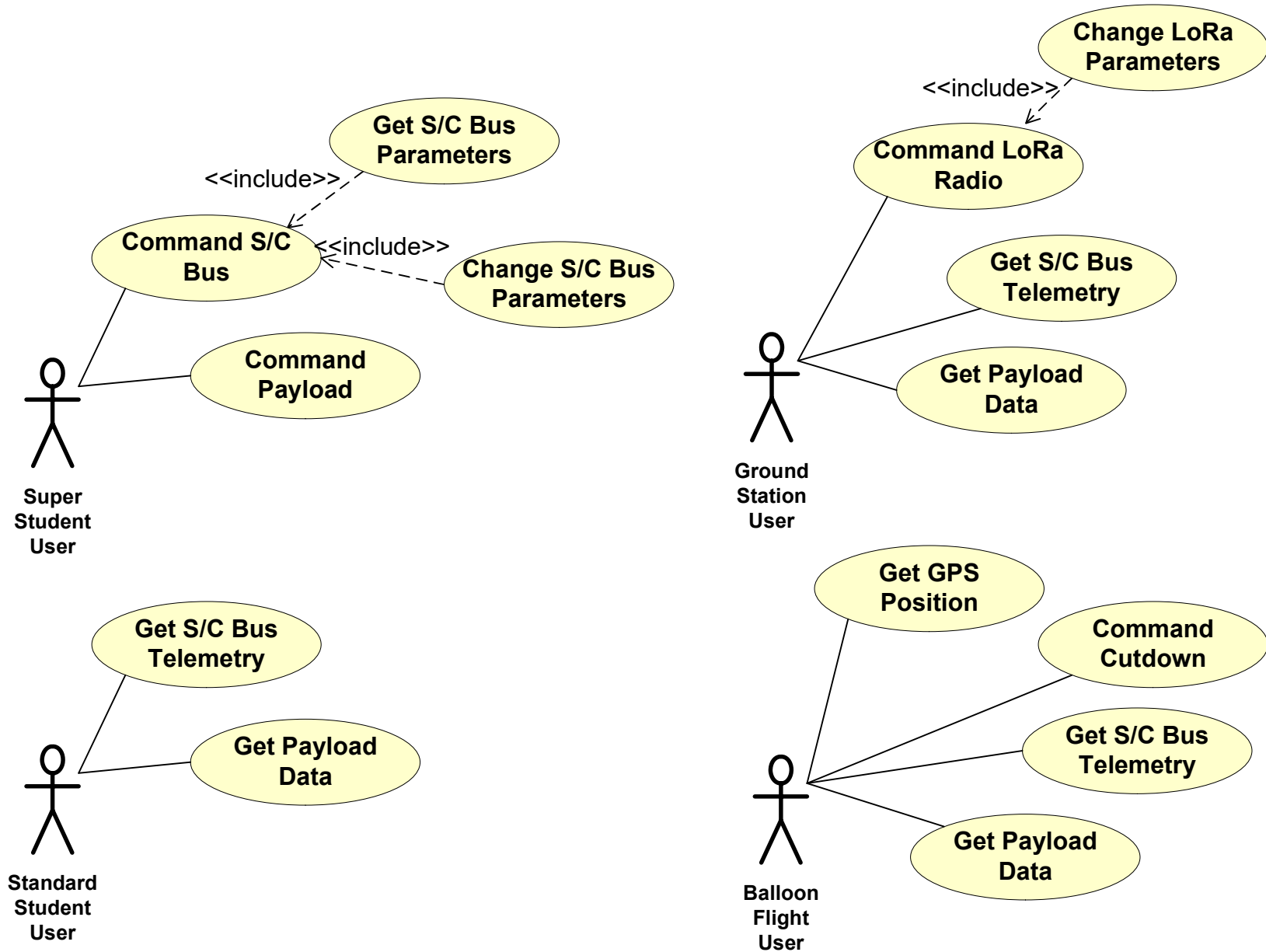


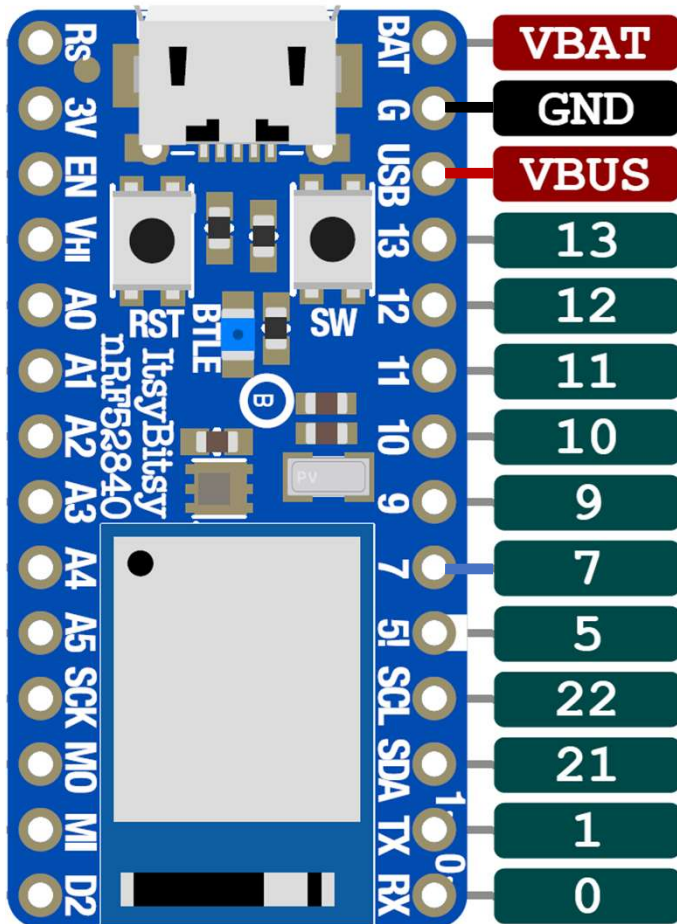
Stick Figures (called Actors) and Bubble Diagrams (called Use Case Diagrams):

- Standard UML (Unified Modeling Language) diagram elements.
- Originally a core component of SW system design
- Now an integral part of SysML, (System Modeling Language) and the system design process (such as that used by INCOSE)

CAPE-Twiggs HSSSI-26 Project

SlimSat User Use Case Analysis





VBAT

GND

VBUS

13

12

11

10

9

7

5

22

21

1

0